

Abhishek Raghav

Btech CSE

CU24250156

Experiment No. 4: Frequency Domain Image Filtering using Fourier Transform

Aim

To implement image filtering in the frequency domain using the Fourier Transform and analyze the effectiveness of frequency-based filtering techniques for noise removal, image enhancement, and feature preservation using Python and OpenCV.

Purpose

Many image processing operations can be performed more efficiently in the frequency domain than in the spatial domain. The Fourier Transform converts an image from the spatial domain into its frequency components, enabling selective manipulation of low-frequency and high frequency information. Frequency domain filtering is widely used in applications such as medical image enhancement, satellite image analysis, image restoration, and biometric systems. In this experiment, students will apply the Discrete Fourier Transform (DFT) to an image, analyze its frequency spectrum, and implement Low-Pass and High-Pass Frequency Filters to observe their effects on image quality and feature extraction.

Algorithm

1. Read the grayscale input image $I(x,y)$.
2. Compute $F(u,v) = \text{DFT}\{I(x,y)\}$.
3. Shift the spectrum so that the DC/zero-frequency component is at the center.
4. Construct a frequency-domain mask $H(u,v)$.
5. For LPF, keep frequencies inside a selected radius and suppress the rest.
6. For HPF, suppress frequencies inside the radius and keep the remaining frequencies.
7. Multiply the shifted spectrum by the filter mask.
8. Shift back and compute the inverse DFT.

9. Normalize the reconstructed result to 0–255.

10. Display and compare the results.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image in grayscale
img = cv2.imread("/content/ChatGPT Image Jun 3, 2026, 01_58_43 PM.png", cv2.IMREAD_GRAYSCALE)

if img is None:
    raise FileNotFoundError("input.jpg not found. Place the image in the same folder")

# 2. Convert image to float32 and compute DFT
img_float = np.float32(img)
dft = cv2.dft(img_float, flags=cv2.DFT_COMPLEX_OUTPUT)

# 3. Shift zero-frequency component to the center
dft_shift = np.fft.fftshift(dft)

# 4. Magnitude spectrum for visualization
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
spectrum = 20 * np.log(magnitude + 1)

rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

# 5. Create Low-Pass and High-Pass masks
radius = 40
y, x = np.ogrid[:rows, :cols]
distance = np.sqrt((x - ccol)**2 + (y - crow)**2)

lpf_mask = np.zeros((rows, cols, 2), np.float32)
lpf_mask[distance <= radius] = 1

hpf_mask = np.ones((rows, cols, 2), np.float32)
hpf_mask[distance <= radius] = 0

# 6. Apply Low-Pass Filter
lpf_shifted = dft_shift * lpf_mask
lpf_ishift = np.fft.ifftshift(lpf_shifted)
lpf_result = cv2.idft(lpf_ishift)
lpf_result = cv2.magnitude(lpf_result[:, :, 0], lpf_result[:, :, 1])
lpf_result = cv2.normalize(lpf_result, None, 0, 255, cv2.NORM_MINMAX)
lpf_result = np.uint8(lpf_result)

# 7. Apply High-Pass Filter
hpf_shifted = dft_shift * hpf_mask
hpf_ishift = np.fft.ifftshift(hpf_shifted)
```

```
hpf_result = cv2.idft(hpf_ishift)
hpf_result = cv2.magnitude(hpf_result[:, :, 0], hpf_result[:, :, 1])
hpf_result = cv2.normalize(hpf_result, None, 0, 255, cv2.NORM_MINMAX)
hpf_result = np.uint8(hpf_result)

# 8. Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(lpf_result, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")

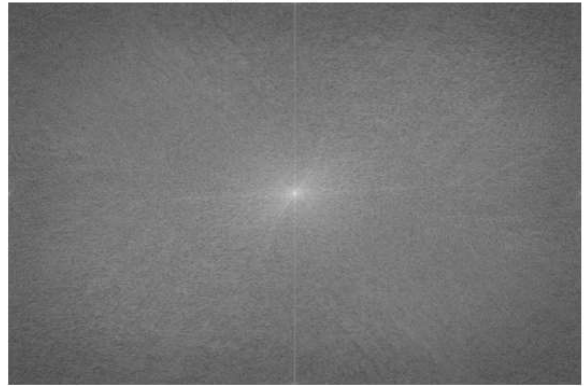
plt.subplot(2, 2, 4)
plt.imshow(hpf_result, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Magnitude Spectrum



Low-Pass Filtered Image



High-Pass Filtered Image



Expected Output

The program displays four images:

- (1) the original grayscale image,
- (2) its Fourier magnitude spectrum,

(3) the Low-Pass Filtered image, and

(4) the High-Pass Filtered image.

The LPF result should appear smoother with reduced fine details, while the HPF result should emphasize edges and fine structures.

Questions

1. What is the Fourier Transform? Why is it important in digital image processing?

Ans 1: The Fourier Transform is a mathematical technique that converts an image from the spatial domain into the frequency domain. It represents an image using different frequency components. It is important for image filtering, noise removal, edge enhancement, image restoration, compression, and frequency analysis.

2. Differentiate between the spatial domain and the frequency domain?

Ans 2: In the spatial domain, operations are performed directly on image pixels and their intensity values. In the frequency domain, operations are performed on frequency components obtained after applying a transform such as the Fourier Transform. Spatial-domain methods are commonly used for local operations such as smoothing and sharpening, whereas frequency-domain methods provide direct control over low- and high-frequency information.

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

Ans 3: The DFT converts a digital image from the spatial domain into the frequency domain and separates it into different frequency components. It is useful for frequency analysis, periodic-noise removal, image filtering, enhancement, restoration, and feature analysis.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

Ans 4: Normally, the zero-frequency component is located at the corners of the Fourier spectrum. `fftshift()` moves it to the center. This makes the spectrum easier to visualize and makes circular Low-Pass and High-Pass filter design more convenient. The center represents low frequencies and regions farther from the center represent higher frequencies.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Ans 5: A Low-Pass Filter allows low frequencies to pass and suppresses high frequencies. It is useful for smoothing and noise reduction. A High-Pass Filter suppresses low frequencies and allows high frequencies to pass, making it useful for edge enhancement, sharpening, and fine-detail extraction.

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

Ans 6: The IDFT converts the filtered frequency-domain representation back into the spatial domain. It reconstructs a normal image after the required frequency components have been modified or removed.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Ans 7: It is preferred when particular frequency components need to be selectively modified. It provides direct control over low and high frequencies and can be effective for periodic noise removal, smoothing, sharpening, image restoration, and other enhancement operations.

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Ans 8: Four applications are: (1) medical imaging for enhancement and restoration of MRI and CT images, (2) satellite imaging for noise removal and feature enhancement, (3) biometric systems for image analysis and feature extraction, and (4) industrial inspection for detecting edges, defects, and surface patterns.

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Ans 9: Spatial-domain filtering works directly on pixels and is generally simple and efficient for small kernels and local operations. Frequency-domain filtering uses transforms and is particularly useful for large filtering operations and problems where selective frequency manipulation is required. Frequency-domain methods are also well suited to periodic noise and advanced image restoration.

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

Ans 10: Frequency-domain filtering allows unwanted frequency components, such as certain noise patterns, to be selectively suppressed while useful information is retained. High-Pass filtering can emphasize edges, boundaries, and fine details, making important features more prominent for subsequent computer-vision and